

Principios S.O.L.I.D.

Tema 1: Introducción a los principios SOLID

¿Qué son los principios SOLID?

Cuando miras hacia atrás, después de haber pasado meses o años corrigiendo el mismo tipo de errores en sistemas que parecían bien diseñados, empiezas a notar patrones. Los sistemas que eran fáciles de modificar al principio se volvían frágiles. Una clase que tocabas por un motivo terminaba rompiendo tres casos de uso que nada tenían que ver. Ahí es donde los principios SOLID entran en escena.

SOLID es un acrónimo que reúne cinco principios de diseño orientado a objetos. No son reglas de obligado cumplimiento, sino directrices que han demostrado su utilidad para evitar que el código se convierta en un lastre con el paso del tiempo.

- **S** – Single Responsibility Principle (SRP)
- **O** – Open/Closed Principle (OCP)
- **L** – Liskov Substitution Principle (LSP)
- **I** – Interface Segregation Principle (ISP)
- **D** – Dependency Inversion Principle (DIP)

Cada uno ataca un síntoma común de degradación del software. Juntos, forman un kit de supervivencia para que el código no muera de éxito.

Contexto: diseño de software orientado a objetos

Para entender SOLID, hay que recordar por qué existe la orientación a objetos. No es solo por tener `class` y `new`. La promesa original era: modelar el dominio en objetos que encapsulan datos y comportamiento, y que se comunican mediante mensajes. Pero enseguida te das cuenta de que si no pones orden, el modelo se convierte en una maraña donde todo depende de todo.

Los principios SOLID nacen de la necesidad de gestionar dos fuerzas opuestas: el cambio (porque los requerimientos evolucionan) y la estabilidad (porque no queremos reescribir todo cada semana). Un diseño que no tiene en cuenta estas fuerzas termina con lo que llamamos "código rígido" (cuesta cambiarlo) y "frágil" (un cambio mínimo lo rompe).

Observa un sistema de gestión de pedidos típico en Python:

```
# Violación tácita de varios principios
class GestorPedidos:
    def __init__(self):
        self.db = Database()
        self.email = EmailSender()

    def procesar(self, pedido):
        total = pedido.calcular_total()
        self.db.guardar(pedido)
```

```
self.email.enviar_factura(pedido.get_cliente(), total)
# ¿y si mañana hay que notificar por SMS también?
# ¿y si el cálculo de total cambia según el país?
```

Cuando ves algo así, sabes que cada nueva exigencia (otro método de envío, otra fuente de persistencia) implicará modificar esta misma clase. SOLID nos da las herramientas para detectar y corregir estas señales de alarma antes de que sea tarde.

Objetivos de aplicar SOLID

A lo largo de los años, he observado que los equipos que aplican consistentemente estos principios obtienen cuatro beneficios clave, aunque al principio parezca que añaden complejidad:

1. Código mantenible

Cuando una clase solo tiene una razón para cambiar (SRP), los cambios se localizan. No tienes que leer quinientas líneas para modificar una regla de negocio. Las modificaciones se sienten como un bisturí, no como un mazo.

2. Código escalable

El principio Abierto/Cerrado (OCP) permite que el sistema crezca añadiendo nuevo código, no modificando el existente. Piensa en un sistema de notificaciones: si diseñas una interfaz **Notificador** y luego implementas **EmailNotificador**, **SMSNotificador**, **PushNotificador**, puedes añadir el centésimo notificador sin tocar los anteriores.

3. Código reutilizable

La segregación de interfaces (ISP) y la inversión de dependencias (DIP) evitan que una clase se vea forzada a depender de métodos que no usa. Cuando las dependencias son hacia abstracciones, puedes reemplazar partes del sistema sin reescribir todo. Es la diferencia entre atornillar un componente y tener un enchufe universal.

4. Código fácil de probar

Una clase que depende de abstracciones (DIP) en lugar de implementaciones concretas se puede probar con dobles de prueba (mocks, stubs). Un ejemplo claro: si tu **ServicioDePagos** depende de una interfaz **PasarelaPago**, puedes pasarle un **PasarelaPagoFalsa** para probar errores sin tener que llamar a una API real. Esto no es un lujo; es una necesidad en sistemas que maduran.

Breve historia: Robert C. Martin y la consolidación de SOLID

No es que Robert C. Martin ("Uncle Bob") inventara estos principios de la nada. El principio de responsabilidad única ya lo mencionaba Tom DeMarco en los 80, el abierto/cerrado lo formalizó Bertrand Meyer en los 90. Lo que hizo Martin fue reunirlos, darles nombres pegadizos y ordenarlos en el acrónimo SOLID a principios de los 2000.

La chispa que lo desencadenó fue la observación directa de cientos de proyectos. Martin notaba que los equipos que seguían ciertas prácticas (sin saberlo) producían sistemas que envejecían bien. El que daba una charla en una conferencia en 2003 expuso por primera vez la lista como "Principles of OOD". Después, Michael Feathers sugirió el acrónimo SOLID porque "es más fácil recordarlo".

Hoy, estos principios se enseñan en casi cualquier curso de arquitectura de software. Pero conviene recordar que no son un dogma. Son el resultado de décadas de fracasos y aciertos. Cuando veas un código que se resiste al cambio, pregúntate cuál de los cinco principios está siendo ignorado. La mayoría de las veces, la respuesta está en el espejo de SOLID.

Ejemplo inicial para abrir el apetito

Imagina una aplicación que genera reportes. Una primera versión ingenua:

```
class GeneradorReportes:
    def __init__(self):
        self.datos = self.leer_base_datos()

    def leer_base_datos(self):
        # conexión a MySQL
        return [1,2,3]

    def formatear(self):
        return "Reporte: " + str(self.datos)

    def enviar_por_email(self):
        print("Enviando...")

# Uso
reporte = GeneradorReportes()
print(reporte.formatear())
reporte.enviar_por_email()
```

¿Qué ocurre cuando quieres cambiar la fuente de datos (de MySQL a una API REST)? Tienes que modificar la clase. ¿Y si quieres enviar el reporte por Slack en lugar de email? Otra modificación. ¿Y si necesitas reutilizar solo el formateo sin la base de datos? Imposible.

Una aplicación temprana de SRP y DIP separaría responsabilidades:

```
from abc import ABC, abstractmethod

class FuenteDatos(ABC):
    @abstractmethod
    def obtener(self):
        pass

class BaseDatosMySQL(FuenteDatos):
    def obtener(self):
```

```

# conexión real
return [1,2,3]

class Formateador(ABC):
    @abstractmethod
    def formatear(self, datos):
        pass

class FormateadorTexto(Formateador):
    def formatear(self, datos):
        return f"Reporte: {datos}"

class Reporte:
    def __init__(self, fuente: FuenteDatos, formateador: Formateador):
        self.fuente = fuente
        self.formateador = formateador

    def generar(self):
        datos = self.fuente.obtener()
        return self.formateador.formatear(datos)

# Ahora puedes inyectar cualquier dependencia
reporte = Reporte(BaseDatosMySQL(), FormateadorTexto())
print(reporte.generar())

```

Las ventajas son inmediatas: la clase **Reporte** no cambia aunque modifiques la fuente o el formato. Cada responsabilidad vive en su propia clase. Esto es SOLID en acción, aunque solo hayamos aplicado parcialmente dos de sus principios.

En los siguientes temas profundizaremos en cada principio por separado, con ejemplos de violaciones reales y refactorizaciones paso a paso.

Tema 2: Principio de Responsabilidad Única (SRP)

Definición: una clase debe tener una sola razón para cambiar

El principio de responsabilidad única suena engañosamente simple: cada clase debería tener una única responsabilidad, un único motivo por el cual alguien la modifique. Pero aquí hay una trampa que ha visto caer a muchos equipos: confundir "hacer una sola cosa" con "tener un único método". Una clase puede tener varios métodos y aun así tener una única responsabilidad si todos ellos sirven a un mismo propósito de alto nivel.

La clave está en la frase "razón para cambiar". Si dos actores diferentes (por ejemplo, el equipo de finanzas y el equipo de logística) pudieran solicitar cambios sobre la misma clase, entonces esa clase tiene más de una responsabilidad. No importa cuán pequeña sea la clase. El acoplamiento entre responsabilidades distintas hará que una modificación por un motivo tenga el potencial de romper la otra funcionalidad.

Síntomas de violación

A lo largo del tiempo, he identificado señales que indican que una clase está gritando por ser dividida:

- **Clases largas:** más de 200 líneas suelen ser un indicio, aunque no siempre. El verdadero problema es cuando la clase tiene múltiples métodos que operan sobre diferentes conjuntos de atributos.
- **Métodos que solo usan un subconjunto de los atributos:** si tienes atributos `total`, `impuestos`, y `direccion_envio`, y algunos métodos solo tocan los primeros dos, es probable que la responsabilidad de envío esté fuera de lugar.
- **Cambios frecuentes por motivos distintos:** mirando el historial de commits, si una clase aparece modificada por tickets de "cambio en cálculo de impuestos" y también por "nuevo formato de reporte", hay una violación.
- **Dificultad para nombrar la clase:** cuando no puedes encontrar un nombre claro que describa qué hace (como "ProcesadorDeTodo"), es una bandera roja.

Ejemplo práctico (antes de refactorizar)

Supongamos un sistema de empleados. La clase `Empleado` maneja datos personales, calcula el salario y también genera un reporte para recursos humanos.

```
class Empleado:
    def __init__(self, nombre, apellido, horas_trabajadas, tarifa_por_hora):
        self.nombre = nombre
        self.apellido = apellido
        self.horas_trabajadas = horas_trabajadas
        self.tarifa_por_hora = tarifa_por_hora

    def nombre_completo(self):
        return f"{self.nombre} {self.apellido}"

    def calcular_salario(self):
        # Lógica que puede cambiar si la empresa modifica las políticas de pago
        return self.horas_trabajadas * self.tarifa_por_hora

    def generar_reporte(self):
        # Formato específico que cambia si RRHH decide otro estilo de documento
        return f"Empleado: {self.nombre_completo()}, Salario: {self.calcular_salario()}"

    def guardar_en_bd(self):
        # Conexión a base de datos
        print(f"Guardando {self.nombre} en la base de datos...")
```

¿Dónde están las responsabilidades mezcladas?

- Cálculo de nómina (responsabilidad del departamento de finanzas)
- Formato de reporte (responsabilidad de RRHH)
- Persistencia (responsabilidad del equipo de infraestructura)

Si finanzas decide cambiar el cálculo a salario base más comisiones, hay que modificar `Empleado`. Si RRHH quiere el reporte en JSON en lugar de texto, otra modificación. Si la base de datos cambia de MySQL a

PostgreSQL, una tercera modificación sobre la misma clase. Tres razones para cambiar → tres responsabilidades.

Refactorización aplicando SRP

Separamos cada responsabilidad en su propia clase. Así, cuando una cambie, las otras no se ven afectadas.

```
# Responsabilidad 1: Datos del empleado
class Empleado:
    def __init__(self, nombre, apellido, horas_trabajadas, tarifa_por_hora):
        self.nombre = nombre
        self.apellido = apellido
        self.horas_trabajadas = horas_trabajadas
        self.tarifa_por_hora = tarifa_por_hora

    def nombre_completo(self):
        return f"{self.nombre} {self.apellido}"

# Responsabilidad 2: Cálculo de salario
class CalculadorSalario:
    @staticmethod
    def calcular(empleado: Empleado):
        # Si la política cambia, solo esta clase se modifica
        return empleado.horas_trabajadas * empleado.tarifa_por_hora

# Responsabilidad 3: Generación de reporte
class GeneradorReporteEmpleado:
    @staticmethod
    def generar(empleado: Empleado):
        salario = CalculadorSalario.calcular(empleado)
        # Si el formato cambia, solo esta clase se actualiza
        return f"Empleado: {empleado.nombre_completo()}, Salario: {salario}"

# Responsabilidad 4: Persistencia
class RepositorioEmpleado:
    @staticmethod
    def guardar(empleado: Empleado):
        # Si cambia la tecnología de BD, solo tocas esta clase
        print(f"Guardando {empleado.nombre} en la base de datos...")

# Uso
emp = Empleado("Ana", "García", 160, 25)
print(GeneradorReporteEmpleado.generar(emp))
RepositorioEmpleado.guardar(emp)
```

Ahora cada clase tiene exactamente una razón para cambiar. El `Empleado` solo cambia si cambian sus atributos o el formato del nombre. `CalculadorSalario` solo cambia por políticas de pago. `GeneradorReporteEmpleado` solo por formato de reporte. `RepositorioEmpleado` solo por infraestructura de persistencia.

Beneficios de aplicar SRP

Cuando se ha aplicado este principio en sistemas reales, los beneficios se notan en el día a día:

- **Menos conflictos en el control de versiones:** dos desarrolladores pueden trabajar en paralelo sobre el cálculo de salario y sobre el reporte sin pisarse, porque modifican archivos diferentes.
- **Pruebas más sencillas:** probar el cálculo de salario de forma aislada es trivial. No necesitas montar una base de datos ni generar reportes falsos para validar la lógica de nómina.
- **Código más legible:** cada clase pequeña cuenta una historia única. No tienes que saltar de un método a otro dentro de un monolito de 500 líneas.
- **Reutilización genuina:** la clase `CalculadorSalario` puede ser usada desde otros contextos (por ejemplo, un servicio de liquidación de sueldos) sin arrastrar dependencias de reporte o persistencia.

Un matiz importante

No hay que caer en el extremo de crear una clase por cada método. El límite es semántico, no mecánico. Si dos métodos colaboran estrechamente para cumplir una misma tarea de alto nivel (por ejemplo, `calcular_bruto()` y `calcular_netto()` dentro de un mismo cálculo de nómina), pueden vivir juntos. La pregunta guía es: "¿Estas funciones cambiarían por la misma razón o por razones distintas?".

Un caso típico de error opuesto es cuando alguien separa `Empleado.nombre` y `Empleado.apellido` en clases diferentes porque "cada atributo es una responsabilidad". Eso es llevarlo al absurdo. La responsabilidad de representar los datos personales del empleado es una sola, aunque tenga varios campos. El SRP busca estabilidad ante cambios de actores, no minimalismo atómico.

En la práctica, aplicar SRP temprano en una refactorización suele ser el primer paso para luego poder aplicar los otros cuatro principios sin dolor. Sin SRP, el resto se vuelve casi imposible.

Tema 3: Principio de Abierto/Cerrado (OCP)

Definición: entidades abiertas para extensión, cerradas para modificación

El principio de abierto/cerrado establece que una entidad de software (clase, módulo, función) debe estar abierta para su extensión pero cerrada para su modificación. Dicho de otro modo: deberías poder añadir nuevo comportamiento sin tener que tocar el código existente que ya funciona.

Cuando escuchas esto por primera vez, parece una contradicción. ¿Cómo vas a añadir algo nuevo sin modificar nada? La respuesta está en el uso de abstracciones. Si diseñas tus clases para que dependan de interfaces o clases base, puedes añadir nuevas implementaciones sin alterar las que ya están probadas y en producción.

La violación más común que he visto es el famoso "if por tipo". Cada vez que aparece un nuevo caso, alguien añade un nuevo `if` o un nuevo `elif` dentro de una función existente. Al principio parece inocente, pero después de diez casos el código se vuelve un infierno de condicionales. Cada nuevo requerimiento significa modificar una función que ya estaba funcionando, con el riesgo de romper los casos anteriores.

Estrategias para implementar OCP

Para lograr que el código esté abierto a extensiones y cerrado a modificaciones, se utilizan principalmente dos mecanismos en la práctica:

1. Herencia y polimorfismo

Defines una clase base o interfaz con métodos abstractos. Luego, cada variante concreta extiende esa base e implementa su propio comportamiento. El cliente que usa la abstracción no necesita cambiar cuando aparece una nueva variante.

2. Composición y estrategia

En lugar de heredar, inyectas dependencias. La clase cliente recibe en su constructor (o mediante un setter) un objeto que implementa cierta interfaz. Para añadir un nuevo comportamiento, solo creas una nueva clase que implemente esa interfaz y la inyectas. La clase cliente permanece intacta.

Ambos enfoques logran lo mismo: el código existente no se modifica, solo se extiende mediante nuevas clases.

Ejemplo de violación (antes)

Supón un sistema de descuentos para una tienda online. Inicialmente hay dos tipos de clientes: regular y premium. El cálculo de descuento se hace mediante condicionales.

```
class CalculadorDescuento:
    def calcular(self, tipo_cliente, monto):
        if tipo_cliente == "regular":
            return monto * 0.05
        elif tipo_cliente == "premium":
            return monto * 0.10
        else:
            return 0

# Uso
calc = CalculadorDescuento()
print(calc.calcular("regular", 1000)) # 50
print(calc.calcular("premium", 1000)) # 100
```

Ahora llega un nuevo requerimiento: clientes "vip" con 20% de descuento. Para añadirlo, tienes que modificar la clase `CalculadorDescuento` añadiendo otro `elif`. Esto viola OCP porque el código existente se modifica. Además, si mañana aparece "empresarial" con 15%, otra modificación. Cada vez que un nuevo tipo de cliente aparece, tocas una clase que ya estaba funcionando. Eventualmente, esa función se llena de condicionales y cualquier cambio puede romper descuentos anteriores.

Refactorización aplicando OCP

Aplicamos el principio usando polimorfismo. Creamos una interfaz (clase abstracta)

`EstrategiaDescuento` y luego implementaciones concretas para cada tipo.


```

from abc import ABC, abstractmethod

class EstrategiaDescuento(ABC):
    @abstractmethod
    def calcular(self, monto):
        pass

class DescuentoRegular(EstrategiaDescuento):
    def calcular(self, monto):
        return monto * 0.05

class DescuentoPremium(EstrategiaDescuento):
    def calcular(self, monto):
        return monto * 0.10

class DescuentoVip(EstrategiaDescuento):
    def calcular(self, monto):
        return monto * 0.20

class DescuentoEmpresarial(EstrategiaDescuento):
    def calcular(self, monto):
        return monto * 0.15

class CalculadorDescuento:
    def __init__(self, estrategia: EstrategiaDescuento):
        self.estrategia = estrategia

    def calcular(self, monto):
        return self.estrategia.calcular(monto)

# Uso
calc_regular = CalculadorDescuento(DescuentoRegular())
print(calc_regular.calcular(1000)) # 50

calc_vip = CalculadorDescuento(DescuentoVip())
print(calc_vip.calcular(1000)) # 200

# Para añadir un nuevo tipo, solo creas una nueva clase
# Sin modificar CalculadorDescuento ni las estrategias existentes

```

Ahora la clase `CalculadorDescuento` está cerrada a modificaciones (no necesitas cambiarla nunca por nuevos tipos de descuento) pero abierta a extensiones (puedes crear nuevas clases que implementen `EstrategiaDescuento`).

Observa un detalle importante: la lógica de selección de qué estrategia usar (el "if" que decidía el tipo) ha desaparecido de la clase de cálculo. Eso no significa que haya desaparecido del sistema, sino que se ha movido a otro lugar, típicamente a una fábrica o al propio código que configura el objeto. Esa es una decisión de diseño: el punto de creación es donde se decide qué estrategia inyectar.

Otro ejemplo: sistema de notificaciones

Imagina un sistema que envía mensajes a usuarios. Una violación típica de OCP:

```
class Notificador:
    def enviar(self, tipo, mensaje, destino):
        if tipo == "email":
            # Lógica de envío de email
            print(f"Enviando email a {destino}: {mensaje}")
        elif tipo == "sms":
            # Lógica de SMS
            print(f"Enviando SMS a {destino}: {mensaje}")
        elif tipo == "whatsapp":
            # Lógica de WhatsApp
            print(f"Enviando WhatsApp a {destino}: {mensaje}")
        # Cada nuevo canal requiere modificar esta clase
```

Cada vez que quieras añadir un nuevo canal (telegram, slack, push notification), tocas **Notificador**. Aplicando OCP:

```
from abc import ABC, abstractmethod

class CanalNotificacion(ABC):
    @abstractmethod
    def enviar(self, mensaje, destino):
        pass

class EmailCanal(CanalNotificacion):
    def enviar(self, mensaje, destino):
        print(f"Enviando email a {destino}: {mensaje}")

class SMSCanal(CanalNotificacion):
    def enviar(self, mensaje, destino):
        print(f"Enviando SMS a {destino}: {mensaje}")

class WhatsAppCanal(CanalNotificacion):
    def enviar(self, mensaje, destino):
        print(f"Enviando WhatsApp a {destino}: {mensaje}")

class Notificador:
    def __init__(self, canal: CanalNotificacion):
        self.canal = canal

    def enviar(self, mensaje, destino):
        self.canal.enviar(mensaje, destino)

# Para añadir Telegram, solo creas:
class TelegramCanal(CanalNotificacion):
    def enviar(self, mensaje, destino):
        print(f"Enviando Telegram a {destino}: {mensaje}")
```

```
# El Notificador original no se toca
```

Patrones relacionados con OCP

En la práctica, OCP se materializa a menudo mediante ciertos patrones de diseño:

- **Strategy Pattern:** exactamente lo que vimos en el ejemplo de descuentos. Define una familia de algoritmos, los encapsula y los hace intercambiables.
- **Template Method Pattern:** defines el esqueleto de un algoritmo en un método de una clase base, y las subclases sobrescriben algunos pasos. El esqueleto permanece cerrado, pero los pasos concretos están abiertos a extensión.
- **Factory Method Pattern:** una clase delega la creación de objetos a subclases, permitiendo extender el tipo de objetos creados sin modificar la clase que los usa.

Un caso real de fallo por no aplicar OCP

Recuerdo un sistema de procesamiento de pagos donde originalmente solo había tarjetas de crédito. La clase `ProcesadorPagos` tenía un método `procesar_tarjeta()`. Luego llegó PayPal, y alguien añadió `procesar_paypal()`. Después llegó criptomonedas, y añadió `procesar_cripto()`. La clase creció a más de 600 líneas con condicionales para elegir qué método llamar según el tipo de pago. Un día, un cambio en el flujo de tarjetas rompió el flujo de PayPal sin que nadie lo notara hasta producción. La lección fue dolorosa: refactorizar a OCP con una interfaz `MetodoPago` y varias implementaciones resolvió el problema de raíz. Ahora cada método de pago vive en su propia clase, aislado de los demás.

Cuándo aplicar OCP (y cuándo no)

Aunque OCP es poderoso, no tiene sentido aplicarlo de forma preventiva en todas partes. Si sabes que solo habrá un tipo de descuento, crear una jerarquía de clases es sobreingeniería. El costo de abstracción (más archivos, más complejidad mental) solo se justifica cuando:

- Esperas que aparezcan múltiples variantes en el futuro (no solo una o dos).
- El área de código cambia con frecuencia por nuevos requerimientos.
- La lógica condicional ya tiene tres o más ramas.

Una estrategia pragmática es comenzar con un código simple (con los `if`) y, cuando aparezca la tercera variante, refactorizar a OCP. La deuda técnica controlada es mejor que la abstracción prematura.

En el siguiente tema veremos cómo el Principio de Sustitución de Liskov garantiza que estas extensiones respeten las expectativas de las clases base.

Tema 4: Principio de Sustitución de Liskov (LSP)

Definición: subtipos deben ser sustituibles por sus tipos base

El principio de sustitución de Liskov, formulado por Barbara Liskov en 1987, dice: si *S* es un subtipo de *T*, entonces los objetos de tipo *T* pueden ser reemplazados por objetos de tipo *S* sin alterar las propiedades

deseables del programa. En términos más llanos: una subclase debe poder usarse en cualquier lugar donde se espere su clase base, y el programa debe seguir comportándose correctamente.

Cuando una subclase viola LSP, el código que depende de la clase base comienza a llenarse de condicionales que preguntan por el tipo real ("si es una instancia de X, haz esto especial"). Eso es una señal inequívoca de que la jerarquía está mal diseñada. El polimorfismo, que debería simplificar el código, se convierte en una fuente de complejidad.

Reglas implícitas de LSP

A lo largo de los años, se han identificado condiciones más precisas que una subclase debe cumplir para no violar LSP:

- **Precondiciones no más fuertes:** una subclase no puede exigir más que su clase base. Si el método base acepta cualquier entero, la subclase no puede restringir a solo enteros positivos.
- **Postcondiciones no más débiles:** la subclase debe garantizar al menos lo que garantiza la base. Si el base asegura devolver un número positivo, la subclase no puede devolver cero o negativo.
- **Invariantes de la clase base deben mantenerse:** si la clase base tiene una invariante (por ejemplo, "el ancho siempre es positivo"), la subclase no debe permitir violarla.
- **Historia o restricción de mutabilidad:** la subclase no debe alterar estados que la base espera que sean inmutables.

El caso clásico de violación que todo desarrollador ha visto al menos una vez es el problema del rectángulo y el cuadrado.

Ejemplo de violación: Rectángulo y Cuadrado

Geométricamente, un cuadrado es un caso particular de rectángulo. Por eso, al modelar en código, es tentador hacer que `Cuadrado` herede de `Rectangulo`. Pero el comportamiento de un cuadrado impone restricciones que un rectángulo no tiene: el ancho y el alto deben ser iguales. Esto rompe LSP.

```
class Rectangulo:
    def __init__(self, ancho, alto):
        self._ancho = ancho
        self._alto = alto

    @property
    def ancho(self):
        return self._ancho

    @ancho.setter
    def ancho(self, valor):
        self._ancho = valor

    @property
    def alto(self):
        return self._alto

    @alto.setter
```

```

def alto(self, valor):
    self._alto = valor

def area(self):
    return self._ancho * self._alto

class Cuadrado(Rectangulo):
    def __init__(self, lado):
        super().__init__(lado, lado)

    @Rectangulo.ancho.setter
    def ancho(self, valor):
        self._ancho = valor
        self._alto = valor # Mantiene la igualdad

    @Rectangulo.alto.setter
    def alto(self, valor):
        self._ancho = valor
        self._alto = valor

# Función que usa Rectangulo
def redimensionar_y_mostrar_area(rect: Rectangulo, nuevo_ancho, nuevo_alto):
    rect.ancho = nuevo_ancho
    rect.alto = nuevo_alto
    print(f"Área esperada: {nuevo_ancho * nuevo_alto}, área real: {rect.area()}")

# Prueba con Rectángulo
r = Rectangulo(2, 3)
redimensionar_y_mostrar_area(r, 4, 5) # Espera 20, obtiene 20. Correcto.

# Prueba con Cuadrado
c = Cuadrado(2)
redimensionar_y_mostrar_area(c, 4, 5) # Espera 20, pero el cuadrado se
convierte en 5x5 = 25

```

El problema: la función `redimensionar_y_mostrar_area` asume que puede modificar ancho y alto independientemente. Con un cuadrado, esa suposición falla. Aunque el código compile, el comportamiento es incorrecto. Un cliente que espera un `Rectangulo` no puede sustituirlo por un `Cuadrado` sin romper su lógica.

¿Por qué falla la intuición?

La trampa aquí es confiar en la relación "es un" del dominio real. En geometría, un cuadrado es un rectángulo. Pero en el comportamiento de software, un cuadrado no es un rectángulo mutable porque la operación "cambiar ancho" en un rectángulo no debería afectar el alto, mientras que en un cuadrado sí. La lección es que la herencia no debe modelar taxonomías del mundo real, sino contratos de comportamiento.

Refactorización: una mejor jerarquía

La solución es no forzar la herencia cuando no hay sustituibilidad. En su lugar, se puede usar una clase base común `Forma` o `Figura` que declare el método `area()`, y que `Rectangulo` y `Cuadrado` implementen independientemente.

```
from abc import ABC, abstractmethod

class Forma(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangulo(Forma):
    def __init__(self, ancho, alto):
        self.ancho = ancho
        self.alto = alto

    def area(self):
        return self.ancho * self.alto

class Cuadrado(Forma):
    def __init__(self, lado):
        self.lado = lado

    def area(self):
        return self.lado ** 2

# Ahora la función funciona con cualquier Forma
def mostrar_area(forma: Forma):
    print(f"Área: {forma.area()}")

r = Rectangulo(4, 5)
c = Cuadrado(4)
mostrar_area(r)  # 20
mostrar_area(c)  # 16
```

Observa que ahora `Rectangulo` y `Cuadrado` no están relacionados por herencia. Comparten la abstracción `Forma`, que define únicamente el comportamiento común que se puede sustituir sin problemas. No hay operaciones de modificación de dimensiones en la interfaz base. Si se necesitaran operaciones de redimensionamiento, habría que diseñarlas con cuidado.

Otro ejemplo común: violación con excepciones

LSP también se aplica a las excepciones lanzadas. Si la clase base declara que lanza `ValueError`, una subclase no puede lanzar `TypeError` porque eso rompería el manejo de errores del cliente.

```
class ProcesadorDatos:
    def procesar(self, datos):
        if not datos:
```

```

        raise ValueError("Datos vacíos")
    return sum(datos)

class ProcesadorDatosLimpio(ProcesadorDatos):
    def procesar(self, datos):
        # Violación: lanza un tipo de excepción diferente
        if not datos:
            raise TypeError("Lista vacía no permitida")
        return sum(datos)

def ejecutar(procesador: ProcesadorDatos, datos):
    try:
        resultado = procesador.procesar(datos)
        print(resultado)
    except ValueError as e:
        print(f"Error manejado: {e}")

# Con ProcesadorDatos funciona, con ProcesadorDatosLimpio falla porque no
# captura TypeError
ejecutar(ProcesadorDatosLimpio(), []) # Excepción no manejada

```

La corrección es que la subclase lance únicamente excepciones que sean subtipos de las que lanza la base (o las mismas). En Python, lanzar `ValueError` está bien; lanzar una excepción no relacionada, no.

El caso de las precondiciones más fuertes

Supón una clase base `CuentaBancaria` con un método `retirar(monto)` que acepta cualquier monto positivo. Una subclase `CuentaAhorros` podría tener un requisito adicional: no se puede retirar más del saldo actual. Pero eso ya es parte del comportamiento esperado. Una precondición más fuerte sería algo como "monto debe ser múltiplo de 10". Eso rompería LSP:

```

class CuentaBancaria:
    def retirar(self, monto):
        if monto <= 0:
            raise ValueError("Monto debe ser positivo")
        # Simulación de retiro
        print(f"Retirando {monto}")

class CuentaAhorros(CuentaBancaria):
    def retirar(self, monto):
        # Precondición más fuerte: solo múltiplos de 10
        if monto % 10 != 0:
            raise ValueError("Monto debe ser múltiplo de 10")
        super().retirar(monto)

def hacer_retiro(cuenta: CuentaBancaria, monto):
    cuenta.retirar(monto)

# Válido con CuentaBancaria, falla con CuentaAhorros
hacer_retiro(CuentaAhorros(), 15) # Lanza error por no ser múltiplo de 10

```

El cliente de **CuentaBancaria** no sabe que debe pasar solo múltiplos de 10. La subclase ha fortalecido la precondition, violando LSP. La solución es relajar esa restricción o elevar la precondition a la base, pero eso afectaría a todos los subtipos.

Consecuencias de ignorar LSP

Cuando se viola LSP, el sistema paga un precio alto:

- **Código cliente lleno de `isinstance`:** para evitar comportamientos incorrectos, los programadores añaden comprobaciones del tipo concreto. Eso destruye el polimorfismo.
- **Pruebas frágiles:** las pruebas escritas para la clase base no funcionan con las subclases, o hay que escribir pruebas especiales.
- **Dificultad de razonar:** ya no puedes confiar en que cualquier objeto de la clase base se comportará de manera consistente. El contrato no se cumple.

Un caso real ocurrió en un sistema de procesamiento de documentos. La clase base **Documento** tenía un método `exportar_pdf()`. Luego se creó **DocumentoImagen** que heredó de **Documento**, pero `exportar_pdf()` lanzaba una excepción porque las imágenes no se exportaban a PDF. El código cliente que esperaba poder exportar cualquier documento fallaba. La solución fue rediseñar: crear una interfaz **ExportableAPDF** y que solo los documentos que realmente soportan PDF la implementen.

Resumen práctico para detectar violaciones

Al revisar una jerarquía de herencia, hazte estas preguntas:

1. ¿El subtipo respeta todas las operaciones del tipo base? ¿Alguna operación se vuelve inválida o sin sentido?
2. ¿El subtipo cambia el comportamiento esperado de alguna manera que pueda sorprender al cliente?
3. ¿El código cliente necesita preguntar por el tipo real para decidir qué hacer?
4. ¿Las pruebas unitarias de la clase base pasan sin modificaciones cuando se ejecutan con el subtipo?

Si la respuesta a alguna es afirmativa, es probable que LSP esté siendo violado. La solución suele ser **preferir la composición sobre la herencia** o rediseñar la jerarquía para que las subclases realmente sean sustituibles.

En el siguiente tema veremos cómo el Principio de Segregación de Interfaces (ISP) ayuda a evitar que las clases tengan que implementar métodos que no necesitan, otro problema común relacionado con contratos.

Tema 5: Principio de Segregación de Interfaces (ISP)

Definición: muchas interfaces específicas mejor que una general

El principio de segregación de interfaces dice que ningún cliente debería depender de métodos que no usa. Dicho de otra forma: es preferible tener varias interfaces pequeñas y específicas, cada una para un

propósito concreto, antes que una única interfaz grande y general que fuerce a las implementaciones a proveer métodos que no necesitan.

Cuando una interfaz es demasiado amplia, cualquier cambio en uno de sus métodos puede afectar a todas las clases que la implementan, aunque no usen ese método. Además, las clases se ven obligadas a implementar comportamientos vacíos o a lanzar excepciones para cumplir con el contrato, lo cual es una señal de mal diseño. La segregación de interfaces es, en cierto sentido, la aplicación del principio de responsabilidad única al nivel de las interfaces.

Síntomas de una interfaz "gorda"

A lo largo del tiempo, se han identificado señales claras de que una interfaz necesita ser segregada:

- **Métodos que lanzan `NotImplementedError` o similares.** Si una clase implementa una interfaz pero varios métodos solo lanzan excepciones, la interfaz es demasiado amplia.
- **Comentarios del tipo "este método no aplica para esta clase"** dentro del código. Es un indicio de que la clase está forzada a cumplir un contrato que no le corresponde.
- **El cliente llama a un subconjunto muy pequeño de los métodos** de la interfaz. Si una interfaz tiene diez métodos y cada cliente solo usa dos o tres, es candidata a ser dividida.
- **Los cambios en un método obligan a modificar clases que ni siquiera lo usan.** Eso ocurre cuando una misma interfaz agrupa responsabilidades distintas.

Ejemplo de violación: trabajadores humanos y robots

Imaginemos un sistema de gestión de empleados. Se define una interfaz `Trabajador` que incluye métodos para comer, dormir y trabajar. Esto funciona para humanos, pero ¿qué pasa cuando se introduce un robot?

```
from abc import ABC, abstractmethod

class Trabajador(ABC):
    @abstractmethod
    def trabajar(self):
        pass

    @abstractmethod
    def comer(self):
        pass

    @abstractmethod
    def dormir(self):
        pass

class Humano(Trabajador):
    def trabajar(self):
        print("Humano trabajando...")

    def comer(self):
        print("Humano comiendo...")
```

```

def dormir(self):
    print("Humano durmiendo...")

class Robot(Trabajador):
    def trabajar(self):
        print("Robot trabajando...")

    def comer(self):
        # Un robot no come. ¿Qué hacemos?
        raise NotImplementedError("Los robots no comen")

    def dormir(self):
        # Los robots no duermen
        raise NotImplementedError("Los robots no duermen")

# Cliente que usa Trabajador
def jornada_completa(trabajador: Trabajador):
    trabajador.trabajar()
    trabajador.comer()
    trabajador.trabajar()
    trabajador.dormir()

humano = Humano()
robot = Robot()
jornada_completa(humano)    # Funciona
jornada_completa(robot)    # Explota con NotImplementedError

```

La interfaz `Trabajador` está mal diseñada porque fuerza a todos los trabajadores a tener comportamientos biológicos. El robot se ve obligado a implementar métodos que no tienen sentido. Esto viola ISP.

Refactorización: segregar interfaces

Dividimos la interfaz grande en tres interfaces más pequeñas: `Trabajable`, `Comible` y `Durmiente`. Luego cada clase implementa solo las que le corresponden.

```

from abc import ABC, abstractmethod

class Trabajable(ABC):
    @abstractmethod
    def trabajar(self):
        pass

class Comible(ABC):
    @abstractmethod
    def comer(self):
        pass

class Durmiente(ABC):

```

```

@abstractmethod
def dormir(self):
    pass

# El humano implementa todo
class Humano(Trabajable, Comible, Durmiente):
    def trabajar(self):
        print("Humano trabajando...")

    def comer(self):
        print("Humano comiendo...")

    def dormir(self):
        print("Humano durmiendo...")

# El robot solo implementa lo que le corresponde
class Robot(Trabajable):
    def trabajar(self):
        print("Robot trabajando...")

# Ahora el cliente puede pedir solo lo que necesita
def jornada_para_humano(trabajador: Trabajable, comible: Comible, durmiente:
Durmiente):
    trabajador.trabajar()
    comible.comer()
    trabajador.trabajar()
    durmiente.dormir()

def jornada_para_robot(trabajador: Trabajable):
    trabajador.trabajar()
    trabajador.trabajar()

humano = Humano()
robot = Robot()
jornada_para_humano(humano, humano, humano) # Funciona
jornada_para_robot(robot) # Funciona sin excepciones

```

Observa que ahora no hay métodos sin implementar. El cliente `jornada_para_humano` exige explícitamente las capacidades que necesita, y el robot simplemente no puede ser pasado donde se espera `Comible` o `Durmiente`. Eso es deseable: el sistema expresa en el sistema de tipos las restricciones reales.

Otro ejemplo: impresoras multifunción

Un ejemplo muy común en la literatura es el de una impresora multifunción. Se define una interfaz `MaquinaOficina` con métodos `imprimir`, `escanear` y `faxear`. Luego hay impresoras simples que solo imprimen, fotocopadoras que imprimen y escanean, etc. La interfaz única obliga a todas a implementar métodos que no soportan.

```

class MaquinaOficina(ABC):
    @abstractmethod
    def imprimir(self, documento):
        pass

    @abstractmethod
    def escanear(self, documento):
        pass

    @abstractmethod
    def enviar_fax(self, documento):
        pass

class ImpresoraSimple(MaquinaOficina):
    def imprimir(self, documento):
        print(f"Imprimiendo: {documento}")

    def escanear(self, documento):
        raise NotImplementedError("No puedo escanear")

    def enviar_fax(self, documento):
        raise NotImplementedError("No puedo enviar fax")

# Cliente que solo quiere imprimir
def realizar_impresion(maquina: MaquinaOficina, doc):
    maquina.imprimir(doc) # Está bien, pero la interfaz exige más

```

Aplicamos segregación:

```

class Imprimible(ABC):
    @abstractmethod
    def imprimir(self, documento):
        pass

class Escaneable(ABC):
    @abstractmethod
    def escanear(self, documento):
        pass

class Faxeable(ABC):
    @abstractmethod
    def enviar_fax(self, documento):
        pass

class ImpresoraSimple(Imprimible):
    def imprimir(self, documento):
        print(f"Imprimiendo: {documento}")

class ImpresoraMultifuncion(Imprimible, Escaneable, Faxeable):
    def imprimir(self, documento):

```

```

        print(f"Imprimiendo: {documento}")

    def escanear(self, documento):
        print(f"Escaneando: {documento}")

    def enviar_fax(self, documento):
        print(f"Enviando fax: {documento}")

# Cliente que solo imprime ahora pide Imprimible
def realizar_impresion(imprimible: Imprimible, doc):
    imprimible.imprimir(doc)

simple = ImpresoraSimple()
multifuncion = ImpresoraMultifuncion()
realizar_impresion(simple, "contrato.pdf")
realizar_impresion(multifuncion, "factura.pdf")

```

Ahora el código expresa exactamente qué capacidades necesita cada cliente. Si en el futuro aparece una `ImpresoraTresD` que también es `Imprimible`, puede integrarse sin problemas.

Un caso real: APIs de persistencia

En un sistema real, se definió una interfaz `Repositorio` con métodos `guardar()`, `actualizar()`, `eliminar()`, `buscar_por_id()`, `buscar_todos()`. Funcionaba bien para bases de datos relacionales. Luego se quiso usar un servicio externo que solo permitía leer datos (API de solo consulta). La implementación de ese repositorio tenía que lanzar excepciones en `guardar`, `actualizar` y `eliminar`. La solución fue segregar en dos interfaces: `RepositorioLectura` (con `buscar_por_id` y `buscar_todos`) y `RepositorioEscritura` (con `guardar`, `actualizar`, `eliminar`). Luego, un cliente que necesitara leer datos se programaba contra `RepositorioLectura`, y uno que necesitara escribir contra `RepositorioEscritura`. Si un repositorio concreto soportaba ambas (como una base de datos), implementaba las dos interfaces.

Relación con otros principios

ISP está muy relacionado con SRP y LSP:

- **ISP complementa a SRP:** si una clase tiene una única responsabilidad, su interfaz pública debería ser pequeña. Si una interfaz es grande, probablemente la clase que la implemente tenga múltiples responsabilidades.
- **ISP evita violaciones de LSP:** si una clase se ve forzada a implementar métodos que no soporta, lanzará excepciones o hará operaciones vacías, rompiendo la sustituibilidad. Segregar interfaces elimina esa necesidad.

Consideraciones prácticas

Segregar interfaces no significa crear una interfaz por cada método. El criterio es agrupar métodos que son **usados juntos** por los mismos clientes. Si un conjunto de métodos siempre se llama en conjunto, pueden estar juntos. Si algunos clientes usan solo un subconjunto, es momento de segregar.

En lenguajes con tipado dinámico como Python, el principio se aplica de forma más flexible. No hay necesidad de declarar explícitamente una interfaz (aunque se puede usar `ABC`). El "contrato" puede ser implícito: una función que espera un objeto con métodos `a()` y `b()` no debería recibir un objeto que tenga además un método `c()` molesto. Pero la violación ocurre igual cuando una clase tiene métodos que no tienen sentido en su contexto. La solución es igualmente válida: dividir las responsabilidades en clases más pequeñas o en "roles" mediante composición.

Un anti-patrón común que he visto es la "interfaz marcadora" (empty interface) para intentar corregir violaciones de ISP. Eso no ayuda. La solución es siempre dividir, no añadir capas vacías.

En el siguiente tema veremos el quinto y último principio: Inversión de Dependencias (DIP), que gobierna cómo las dependencias deben apuntar hacia abstracciones y no hacia concreciones.

Tema 6: Principio de Inversión de Dependencias (DIP)

Definición: depender de abstracciones, no de concreciones

El principio de inversión de dependencias establece dos premisas fundamentales:

1. Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones.
2. Las abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones.

Dicho de forma más directa: no quiero que la lógica central de mi negocio (el corazón de la aplicación) esté atada a detalles concretos como una base de datos específica, un framework web o una librería de envío de correos. Si esos detalles cambian (y siempre cambian), el corazón de la aplicación debería permanecer intacto. Para lograrlo, el módulo de alto nivel define una abstracción (una interfaz) y el módulo de bajo nivel implementa esa abstracción. Así, la dirección de la dependencia se invierte: ahora el detalle concreto depende de la abstracción que el dominio necesita, y no al revés.

La confusión habitual es pensar que "inversión de dependencias" es lo mismo que "inyección de dependencias". La inyección es una técnica para aplicar el principio, pero no es el principio en sí. El principio habla de la dirección de las dependencias en el diseño, no de cómo se construyen los objetos.

Síntomas de violación de DIP

Una década corrigiendo sistemas me ha enseñado a reconocer estas señales:

- **Clases de alto nivel importando directamente clases concretas** (por ejemplo, `import mysql.connector` dentro de un servicio de facturación).
- **Cambios en la base de datos o en un servicio externo que obligan a modificar la lógica de negocio.** Si un cambio de MySQL a PostgreSQL implica tocar tu capa de dominio, DIP está violado.
- **Dificultad para probar unidades de negocio en aislamiento.** Si necesitas una base de datos real o un servidor SMTP para probar una regla de negocio, las dependencias están mal dirigidas.
- **El código de alto nivel sabe demasiado sobre los detalles de bajo nivel**, por ejemplo, construyendo cadenas de conexión, manejando excepciones específicas de la librería, o llamando a métodos muy particulares del detalle.

Ejemplo de violación (antes)

Imaginemos un sistema de autenticación que verifica usuarios contra una base de datos MySQL. El servicio de autenticación es un módulo de alto nivel (la política de negocio), pero depende directamente del concreto `RepositorioUsuarioMySQL`.

```
# Módulo de bajo nivel (detalle)
class RepositorioUsuarioMySQL:
    def __init__(self):
        # Conexión real a MySQL
        self.conexion = self._conectar()

    def _conectar(self):
        print("Conectando a MySQL...")
        return "conexion_mysql"

    def buscar_por_email(self, email):
        print(f"SELECT * FROM usuarios WHERE email = '{email}'")
        if email == "admin@ejemplo.com":
            return {"email": email, "clave_hash": "hash_secreto"}
        return None

# Módulo de alto nivel (lógica de negocio)
class ServicioAutenticacion:
    def __init__(self):
        # Dependencia directa de una clase concreta (violación)
        self.repo = RepositorioUsuarioMySQL()

    def autenticar(self, email, password):
        usuario = self.repo.buscar_por_email(email)
        if not usuario:
            return False
        # Verificar password (simplificado)
        return password == "secreto" # solo para ejemplo

# Uso
auth = ServicioAutenticacion()
print(auth.autenticar("admin@ejemplo.com", "secreto"))
```

¿Qué ocurre cuando se quiere cambiar a PostgreSQL? Hay que modificar `ServicioAutenticacion` para que use un `RepositorioUsuarioPostgreSQL`. ¿Y si la autenticación pasa a ser contra un servicio LDAP? Otra modificación. El módulo de alto nivel está acoplado a detalles concretos, lo que viola DIP.

Refactorización aplicando DIP

Creamos una abstracción (interfaz) para el repositorio de usuarios. El `ServicioAutenticacion` depende de esa abstracción, y las implementaciones concretas (MySQL, PostgreSQL, LDAP) dependen también de la abstracción.

```

from abc import ABC, abstractmethod

# Abstracción (interfaz) definida en el módulo de alto nivel
class RepositorioUsuario(ABC):
    @abstractmethod
    def buscar_por_email(self, email):
        pass

# Módulo de bajo nivel (detalle) implementa la abstracción
class RepositorioUsuarioMySQL(RepositorioUsuario):
    def buscar_por_email(self, email):
        print(f"SELECT * FROM usuarios WHERE email = '{email}' (MySQL)")
        if email == "admin@ejemplo.com":
            return {"email": email, "clave_hash": "hash_secreto"}
        return None

# Otro detalle: PostgreSQL
class RepositorioUsuarioPostgreSQL(RepositorioUsuario):
    def buscar_por_email(self, email):
        print(f"SELECT * FROM usuarios WHERE email = '{email}' (PostgreSQL)")
        if email == "admin@ejemplo.com":
            return {"email": email, "clave_hash": "hash_secreto"}
        return None

# Módulo de alto nivel: ahora depende de la abstracción, no de una concreción
class ServicioAutenticacion:
    def __init__(self, repo: RepositorioUsuario): # Inyección de dependencia
        self.repo = repo

    def autenticar(self, email, password):
        usuario = self.repo.buscar_por_email(email)
        if not usuario:
            return False
        return password == "secreto"

# Uso: la dependencia se inyecta desde fuera
repo_mysql = RepositorioUsuarioMySQL()
auth = ServicioAutenticacion(repo_mysql)
print(auth.autenticar("admin@ejemplo.com", "secreto"))

# Cambiar a PostgreSQL sin tocar ServicioAutenticacion
repo_pg = RepositorioUsuarioPostgreSQL()
auth2 = ServicioAutenticacion(repo_pg)
print(auth2.autenticar("admin@ejemplo.com", "secreto"))

```

Ahora el `ServicioAutenticacion` no sabe nada de MySQL ni de PostgreSQL. Solo conoce la interfaz `RepositorioUsuario`. Para probar la autenticación, podemos inyectar un repositorio falso que devuelva valores controlados. La dirección de la dependencia se ha invertido: antes la lógica de negocio dependía del detalle; ahora ambos dependen de la abstracción.

Otro ejemplo: sistema de notificaciones con múltiples canales

Un caso recurrente es un servicio que envía notificaciones. Violación típica:

```
class Notificador:
    def __init__(self):
        self.email_cliente = EmailCliente() # Concreto
        self.sms_cliente = SMSCliente()     # Concreto

    def enviar_bienvenida(self, usuario):
        self.email_cliente.enviar(usuario.email, "Bienvenido")
        self.sms_cliente.enviar(usuario.telefono, "Bienvenido")
```

Para aplicar DIP, definimos una abstracción **Mensajero**:

```
from abc import ABC, abstractmethod

class Mensajero(ABC):
    @abstractmethod
    def enviar(self, destino, mensaje):
        pass

class EmailMensajero(Mensajero):
    def enviar(self, destino, mensaje):
        print(f"Enviando email a {destino}: {mensaje}")

class SMSMensajero(Mensajero):
    def enviar(self, destino, mensaje):
        print(f"Enviando SMS a {destino}: {mensaje}")

class Notificador:
    def __init__(self, mensajero: Mensajero):
        self.mensajero = mensajero

    def enviar_bienvenida(self, usuario):
        self.mensajero.enviar(usuario.get_contacto(), "Bienvenido")

# Uso
usuario = type('Usuario', (), {'get_contacto': lambda: "email@example.com"})()
notif = Notificador(EmailMensajero())
notif.enviar_bienvenida(usuario)
```

Ahora **Notificador** puede reutilizarse con cualquier canal de mensajería, y la lógica de negocio ("enviar bienvenida") no se contamina con detalles de envío.

La pirámide de dependencias

Una imagen mental que ayuda es pensar en la arquitectura como un triángulo: en la cima están los módulos de alto nivel (políticas, reglas de negocio) que son estables y cambian poco. En la base están los detalles (base de datos, UI, frameworks) que cambian frecuentemente. DIP dice que las flechas de dependencia deben apuntar hacia la cima, es decir, los detalles deben depender de las políticas, nunca al revés. Sin DIP, la flecha apunta hacia abajo y la base inestable contamina la cima.

Inyección de dependencias vs. DIP

A menudo se confunden ambos conceptos. La inyección de dependencias es un mecanismo (pasar dependencias por constructor, setter o interfaz) que facilita aplicar DIP. Pero es perfectamente posible tener inyección sin DIP (si inyectas una clase concreta en lugar de una abstracción). También es posible tener DIP sin inyección (por ejemplo, usando un contenedor de servicios que resuelva la abstracción automáticamente). En la práctica, la inyección de dependencias es la técnica más común para cumplir DIP.

Un caso real: migración de almacenamiento

En un sistema de gestión de archivos, el módulo de procesamiento de documentos dependía directamente de una librería de S3 (Amazon). Cuando se necesitó migrar a Azure Blob Storage, el equipo tuvo que modificar el procesador de documentos porque este construía rutas y credenciales específicas de S3. La refactorización introdujo una abstracción `AlmacenamientoArchivos` con métodos `guardar()`, `recuperar()`, `eliminar()`. El procesador de documentos pasó a depender de esa interfaz, y se implementaron dos adaptadores: `S3Almacenamiento` y `AzureAlmacenamiento`. La migración fue limpia, y las pruebas unitarias dejaron de necesitar una cuenta real en la nube.

Cuándo aplicar DIP (y cuándo no)

No tiene sentido aplicar DIP para cada dependencia trivial, especialmente si es parte del mismo módulo o si es una clase de utilidad que se sabe que no cambiará (por ejemplo, `datetime.datetime`). Tampoco conviene añadir interfaces para cada clase solo por seguir el principio. La clave es identificar **fronteras arquitectónicas**: lo que cambia por razones distintas a la lógica de negocio.

Las dependencias que merecen abstracción suelen ser:

- Bases de datos y sistemas de persistencia.
- Servicios externos (APIs, colas de mensajes, sistemas de archivos).
- Frameworks de UI o de infraestructura.
- Librerías que pueden ser reemplazadas (envío de correos, logging, caché).

Relación con los otros principios

DIP es el cierre de SOLID. Los cuatro principios anteriores preparan el terreno para que DIP sea aplicable:

- **SRP** asegura que cada clase tenga una sola razón para cambiar, lo que hace más fácil definir abstracciones centradas.
- **OCP** permite extender comportamiento sin modificar, y DIP proporciona el mecanismo de abstracciones para lograr eso.
- **LSP** garantiza que las implementaciones concretas puedan sustituirse por las abstracciones sin romper el sistema.

- **ISP** evita que las abstracciones sean demasiado grandes, lo que facilita implementar DIP porque las interfaces pequeñas son más estables.

Aplicados juntos, SOLID permite construir sistemas donde el dominio del negocio está desacoplado de los detalles técnicos. El resultado es código que resiste la prueba del tiempo: los cambios en infraestructura no sacuden la lógica central, y la lógica central se puede probar, entender y modificar con confianza.

Tema 7: Aplicación conjunta de los principios SOLID

Relaciones y sinergias entre los cinco principios

Los cinco principios no son islas independientes. Se refuerzan mutuamente y, aplicados en conjunto, producen un efecto multiplicador. Cuando has trabajado el tiempo suficiente con código que los viola, empiezas a ver un patrón: una violación de SRP suele llevar a violaciones de OCP y DIP. Una jerarquía que no respeta LSP acaba forzando interfaces gordas que violan ISP. Cada principio tapa un agujero que los otros dejan, y juntos forman una red de contención.

- **SRP + ISP**: si una clase tiene una sola responsabilidad, sus interfaces tienden a ser pequeñas. ISP fuerza a que las interfaces sean específicas, lo que a su vez facilita que las clases tengan una sola razón para cambiar.
- **OCP + DIP**: OCP dice "abierto a extensión, cerrado a modificación". DIP da el mecanismo: depende de abstracciones, no de concreciones. Sin DIP, OCP es casi imposible de cumplir porque cualquier extensión requeriría modificar dependencias directas.
- **LSP + ISP**: cuando las interfaces son pequeñas y específicas, es mucho más fácil que las implementaciones respeten la sustitución. Interfaces grandes fuerzan a las subclasses a implementar métodos que no necesitan, lo que casi siempre rompe LSP.
- **Todos juntos**: un sistema que cumple SRP, OCP, LSP, ISP y DIP es un sistema donde cada componente tiene una responsabilidad clara, se puede extender sin tocar lo existente, las sustituciones son seguras, las interfaces son ajustadas y las dependencias apuntan hacia abstracciones estables.

Anti-patrones comunes que SOLID ayuda a evitar

A lo largo de los años, he visto los mismos anti-patrones una y otra vez. SOLID proporciona un antídoto para cada uno:

1. **God Object (Objeto Dios)**: una clase que hace demasiado. → SRP dice que la divides.
2. **Switch Statement Smell**: un bloque `if-elif-else` o `switch` que crece sin control. → OCP sugiere polimorfismo y estrategias.
3. **Instanceof/Type Checking**: código que pregunta "¿qué tipo eres?" para decidir qué hacer. → LSP garantiza que no sea necesario.
4. **Fat Interface**: interfaces enormes que obligan a implementar métodos vacíos. → ISP fuerza a segregar.
5. **Hard-coded Dependencies**: `new` dentro de una clase de alto nivel. → DIP dice que dependas de abstracciones y las inyectes.

Ejemplo completo: refactorización paso a paso

Imaginemos un sistema de gestión de pedidos que ha crecido orgánicamente. Partimos de una clase monolítica que viola todos los principios. Luego la refactorizamos aplicando SOLID de forma incremental.

Versión inicial (todo violado)

```
class GestorPedidos:
    def __init__(self):
        self.db_connection = "mysql://localhost" # Detalle concreto
        self.smtp_server = "smtp.gmail.com"

    def calcular_total(self, pedido):
        # Responsabilidad: cálculo (pero mezclado con otras)
        total = 0
        for item in pedido["items"]:
            total += item["precio"] * item["cantidad"]
        # Descuento solo para clientes especiales
        if pedido.get("cliente_tipo") == "vip":
            total *= 0.9
        return total

    def guardar_pedido(self, pedido):
        # Responsabilidad: persistencia
        print(f"Guardando en {self.db_connection}: {pedido}")

    def enviar_confirmacion(self, pedido):
        # Responsabilidad: notificación
        print(f"Enviando email desde {self.smtp_server} a {pedido['email']}")

    def procesar(self, pedido):
        total = self.calcular_total(pedido)
        pedido["total"] = total
        self.guardar_pedido(pedido)
        self.enviar_confirmacion(pedido)
        return total

# Uso
pedido = {
    "items": [{"precio": 100, "cantidad": 2}],
    "cliente_tipo": "vip",
    "email": "cliente@ejemplo.com"
}
gestor = GestorPedidos()
print(gestor.procesar(pedido))
```

Problemas:

- SRP: la clase tiene al menos tres responsabilidades (cálculo, persistencia, notificación).
- OCP: para añadir un nuevo método de envío (SMS) o nueva base de datos, hay que modificar `GestorPedidos`.

- LSP: no aplica directamente porque no hay herencia, pero el diseño impide cualquier extensión polimórfica.
- ISP: no hay interfaces.
- DIP: depende de concreciones (cadena de conexión, servidor SMTP).

Paso 1: Aplicar SRP - Separar responsabilidades

Creamos clases separadas para cálculo, persistencia y notificación.

```
class CalculadorTotal:
    def calcular(self, pedido):
        total = 0
        for item in pedido["items"]:
            total += item["precio"] * item["cantidad"]
        if pedido.get("cliente_tipo") == "vip":
            total *= 0.9
        return total

class RepositorioPedidos:
    def guardar(self, pedido):
        print(f"Guardando en mysql://localhost: {pedido}")

class NotificadorPedidos:
    def enviar(self, pedido):
        print(f"Enviando email desde smtp.gmail.com a {pedido['email']}")

class GestorPedidos:
    def __init__(self):
        self.calculador = CalculadorTotal()
        self.repositorio = RepositorioPedidos()
        self.notificador = NotificadorPedidos()

    def procesar(self, pedido):
        total = self.calculador.calcular(pedido)
        pedido["total"] = total
        self.repositorio.guardar(pedido)
        self.notificador.enviar(pedido)
        return total
```

Ahora cada clase tiene una responsabilidad, pero `GestorPedidos` sigue dependiendo de clases concretas (viola DIP).

Paso 2: Aplicar DIP e ISP - Definir abstracciones

Creamos interfaces (clases abstractas) para cada colaborador, y hacemos que `GestorPedidos` dependa de ellas.

```

from abc import ABC, abstractmethod

# Abstracciones
class CalculadorTotalInterface(ABC):
    @abstractmethod
    def calcular(self, pedido):
        pass

class RepositorioPedidosInterface(ABC):
    @abstractmethod
    def guardar(self, pedido):
        pass

class NotificadorPedidosInterface(ABC):
    @abstractmethod
    def enviar(self, pedido):
        pass

# Implementaciones concretas (detalles)
class CalculadorTotalEstandar(CalculadorTotalInterface):
    def calcular(self, pedido):
        total = 0
        for item in pedido["items"]:
            total += item["precio"] * item["cantidad"]
        if pedido.get("cliente_tipo") == "vip":
            total *= 0.9
        return total

class RepositorioPedidosMySQL(RepositorioPedidosInterface):
    def guardar(self, pedido):
        print(f"Guardando en MySQL: {pedido}")

class NotificadorPedidosEmail(NotificadorPedidosInterface):
    def enviar(self, pedido):
        print(f"Enviando email a {pedido['email']}")

# Módulo de alto nivel ahora depende de abstracciones
class GestorPedidos:
    def __init__(self,
                 calculador: CalculadorTotalInterface,
                 repositorio: RepositorioPedidosInterface,
                 notificador: NotificadorPedidosInterface):
        self.calculador = calculador
        self.repositorio = repositorio
        self.notificador = notificador

    def procesar(self, pedido):
        total = self.calculador.calcular(pedido)
        pedido["total"] = total
        self.repositorio.guardar(pedido)
        self.notificador.enviar(pedido)
        return total

```

Ya tenemos DIP. Además, las interfaces son pequeñas (ISP). Ahora podemos inyectar cualquier implementación.

Paso 3: Aplicar OCP - Permitir nuevas estrategias sin modificar

Ahora queremos añadir nuevos tipos de cálculo de descuento (no solo VIP). Podemos extender sin modificar `GestorPedidos` aplicando OCP. Creamos una jerarquía de estrategias de descuento.

```
class EstrategiaDescuento(ABC):
    @abstractmethod
    def aplicar(self, total, pedido):
        pass

class SinDescuento(EstrategiaDescuento):
    def aplicar(self, total, pedido):
        return total

class DescuentoVIP(EstrategiaDescuento):
    def aplicar(self, total, pedido):
        return total * 0.9

class DescuentoBlackFriday(EstrategiaDescuento):
    def aplicar(self, total, pedido):
        return total * 0.8

class CalculadorTotalConDescuento(CalculadorTotalInterface):
    def __init__(self, estrategia: EstrategiaDescuento):
        self.estrategia = estrategia

    def calcular(self, pedido):
        total = 0
        for item in pedido["items"]:
            total += item["precio"] * item["cantidad"]
        return self.estrategia.aplicar(total, pedido)

# Ahora podemos construir el gestor con diferentes estrategias
estrategia = DescuentoVIP()
calculador = CalculadorTotalConDescuento(estrategia)
repositorio = RepositorioPedidosMySQL()
notificador = NotificadorPedidosEmail()

gestor = GestorPedidos(calculador, repositorio, notificador)
```

OCP cumplido: añadimos `DescuentoBlackFriday` sin tocar nada existente.

Paso 4: Asegurar LSP en la jerarquía de descuentos

¿Podríamos tener una subclase de `EstrategiaDescuento` que imponga precondiciones? Por ejemplo, `DescuentoPorMontoMinimo` que requiere que el total supere 100. Si un cliente espera cualquier `EstrategiaDescuento` y le pasamos esta, podría fallar inesperadamente. Para respetar LSP, la estrategia debe manejar el caso contrario sin romper el contrato:

```
class DescuentoPorMontoMinimo(EstrategiaDescuento):
    def __init__(self, monto_minimo, porcentaje):
        self.monto_minimo = monto_minimo
        self.porcentaje = porcentaje

    def aplicar(self, total, pedido):
        if total >= self.monto_minimo:
            return total * (1 - self.porcentaje)
        return total # No aplica descuento, pero no lanza error ni viola el
contrato
```

Así cualquier cliente de `EstrategiaDescuento` recibe siempre un número; el comportamiento es predecible. LSP respetado.

Paso 5: Verificar ISP en las interfaces

Las interfaces que definimos (`CalculadorTotalInterface`, `RepositorioPedidosInterface`, `NotificadorPedidosInterface`) tienen un solo método cada una. No hay métodos que una implementación se vea forzada a dejar vacíos. ISP se cumple naturalmente.

Resultado final

El sistema refactorizado:

- Cada clase tiene una sola responsabilidad (SRP).
- Se puede extender añadiendo nuevas estrategias, repositorios o notificadores sin modificar `GestorPedidos` (OCP).
- Cualquier implementación de `EstrategiaDescuento` es sustituible por otra sin romper el contrato (LSP).
- Las interfaces son pequeñas y específicas (ISP).
- Las dependencias apuntan hacia abstracciones, y los detalles concretos dependen de esas abstracciones (DIP).

Beneficios observables en el día a día

Cuando un equipo aplica SOLID de forma consistente, la diferencia se nota:

- **Los pull requests se vuelven más pequeños:** cada cambio suele afectar a una sola clase o a una nueva implementación de una interfaz. El riesgo de romper funcionalidades no relacionadas disminuye drásticamente.
- **Las pruebas unitarias se escriben una sola vez:** puedes probar `GestorPedidos` con mocks de sus dependencias en milisegundos. Las pruebas de integración quedan solo para verificar el ensamblaje

final.

- **Los nuevos miembros del equipo entienden el código más rápido:** porque cada clase cuenta una historia clara. El sobreesfuerzo inicial de abstracción se paga con creces en legibilidad.
- **El sistema sobrevive a cambios de infraestructura:** migrar de MySQL a PostgreSQL o de email a Slack implica solo escribir una nueva implementación de la interfaz correspondiente y cambiar la inyección en un solo lugar (el punto de composición, como el `main` o un contenedor de DI).

Limitaciones y crítica final

No todo es perfecto. Aplicar SOLID de manera dogmática puede llevar a una "explosión de clases". Un sistema con cinco principios puede tener decenas de archivos pequeños, lo que a veces abruma. El truco está en aplicarlos **en las fronteras que sí cambian**. No hace falta abstraer cada suma de dos números. La experiencia enseña a identificar qué partes del sistema son volátiles y merecen el esfuerzo.

También hay lenguajes (como Python) donde algunos principios se aplican de forma más laxa gracias al tipado pato. Pero los beneficios conceptuales siguen siendo los mismos: código desacoplado, testeable y mantenible. Al final, SOLID no es una checklist, es una brújula. Cuando te enfrentas a un código que resiste al cambio, repasa cada principio y pregunta: "¿Esto está violando alguno?" Casi siempre la respuesta te indicará el camino.

En los temas siguientes (o en la práctica diaria) aplica estos principios con criterio. El mejor sistema no es el que más principios cumple, sino el que permite añadir nuevas funcionalidades con confianza y sin miedo a romper lo que ya funciona.

Tema 8: Limitaciones y críticas

Contextos donde SOLID puede ser excesivo

Aplicar SOLID en cada línea de código es como usar un martillo pilón para clavar una chincheta. Hay contextos donde los cinco principios añaden más complejidad que valor real. El primer aviso es cuando el equipo pasa más tiempo diseñando jerarquías de interfaces que escribiendo lógica de negocio.

- **Proyectos pequeños o scripts de una sola ejecución:** un script de 100 líneas que procesa un archivo CSV y envía un correo no necesita una fábrica de estrategias de descuento ni inyección de dependencias. Los principios SOLID tienen un costo de entrada en número de archivos y abstracciones que, para un script desechable o un prototipo, es difícil de justificar. He visto equipos perder horas diseñando la interfaz `LectorArchivo` para luego implementar `LectorCSV`, `LectorJSON` y `LectorExcel`, cuando el requerimiento jamás pidió más que CSV.
- **Código de infraestructura muy estable:** una librería de utilidades matemáticas que no cambiará en años, con funciones puras, no necesita ser abierta a extensión mediante polimorfismo. Un simple `if` dentro de una función puede ser perfectamente legible y más rápido.
- **Proyectos con fecha de caducidad corta:** si sabes que el software será reemplazado en seis meses, invertir en un diseño desacoplado es una apuesta pobre. El principio de "you ain't gonna need it" (YAGNI) pesa más que SOLID.

Costo inicial de diseño vs beneficio a largo plazo

Hay un trade-off inevitable: aplicar SOLID desde el principio requiere más tiempo de diseño, más archivos, más decisiones sobre abstracciones. Ese costo se paga por adelantado. El beneficio (cambios más fáciles, menos errores) se cosecha después del tercer o cuarto ciclo de cambios.

La curva es conocida: sin SOLID, las primeras semanas son muy rápidas. Llega un punto (el "punto de quiebre") donde la deuda técnica hace que cada cambio tarde el doble. Con SOLID, el arranque es más lento, pero la pendiente de degradación es mucho más suave. El error que he visto repetirse es aplicar SOLID solo cuando ya es demasiado tarde, cuando la clase de 3000 líneas ya está en producción. Refactorizar bajo presión es más caro que diseñar bien al principio, pero también es más caro que no aplicar SOLID cuando realmente no se necesita.

Críticas académicas y prácticas

El mundo del software no es unánime. Varios autores y profesionales han señalado problemas con SOLID:

1. **Sobrediseño y complejidad accidental:** crear una interfaz para cada dependencia puede triplicar el número de archivos. En lenguajes como Java o C#, eso es aceptado. En Python, muchos equipos prefieren el "duck typing" y consideran que las clases abstractas son opcionales. La crítica es que SOLID puede llevar a un estilo de programación "industrial" innecesariamente pesado.
2. **El principio de sustitución de Liskov es difícil de verificar automáticamente:** no hay un compilador que garantice que una subclase no fortalezca precondiciones. En la práctica, las violaciones de LSP suelen detectarse en pruebas o en producción. Algunos críticos argumentan que LSP es más un ideal que una regla aplicable sistemáticamente.
3. **OCP puede fomentar la proliferación de clases:** cada nueva variante requiere una nueva clase. Si las variantes son muchas y simples (por ejemplo, diez tipos de descuento con una línea de diferencia), terminas con diez archivos de cuatro líneas cada uno. Alternativas como pasar una función lambda o usar diccionarios de configuración pueden ser más concisas.
4. **SRP es subjetivo:** ¿qué es "una razón para cambiar"? Eso depende de la organización, de los actores, del contexto. Dos equipos pueden interpretar la misma clase de forma distinta. Algunos críticos señalan que SRP es útil como principio heurístico, pero no como una métrica precisa.
5. **DIP añade indirección:** cada abstracción es una capa de indirección. Demasiada indirección dificulta seguir el flujo del programa. Un desarrollador nuevo puede tener que saltar entre cinco archivos para entender una operación simple.

El problema de la abstracción prematura

Hay un patrón recurrente: alguien lee SOLID, se entusiasma, y aplica DIP y OCP a cada posible variante futura. Crea `InterfaceUsuario`, `InterfaceRepositorio`, `InterfaceServicio`, y luego resulta que solo hay una implementación real durante años. Esa abstracción nunca fue necesaria. El costo de mantenerla (renombrar métodos, entender la indirección) es un impuesto eterno por un beneficio que nunca llegó.

La alternativa pragmática es la **abstracción diferida**: empieza con una clase concreta. Cuando aparezca el segundo caso que requeriría un condicional o una duplicación, entonces extrae una interfaz y aplica OCP. Hasta entonces, el código simple es mejor. He aplicado esta estrategia en decenas de proyectos y el resultado es menos complejidad accidental sin sacrificar la capacidad de evolución.

SOLID no es suficiente (ni pretende serlo)

SOLID resuelve problemas de diseño a nivel de clases y módulos pequeños. No aborda arquitectura de sistemas completos (microservicios, eventos, CQRS, etc.). Tampoco cubre temas como concurrencia, manejo de errores transaccionales, o rendimiento. Un sistema puede cumplir SOLID al pie de la letra y ser lento, inseguro o no escalar horizontalmente.

Además, SOLID asume un estilo orientado a objetos con herencia y polimorfismo. En paradigmas funcionales, muchos de estos principios se cumplen de forma natural sin necesidad de clases (por ejemplo, OCP se logra con funciones de orden superior, sin herencia). Forzar SOLID en un estilo funcional puede ser contraproducente.

Casos reales donde SOLID falló como guía

Recuerdo un sistema embebido con memoria limitada y requisitos de tiempo real. Aplicar SRP y DIP llevó a una explosión de objetos pequeños y llamadas virtuales que degradaron el rendimiento y aumentaron el consumo de memoria. La solución fue reescribir varios módulos en un estilo más monolítico pero con acceso directo a datos. Las clases grandes, con métodos que violaban SRP, eran más eficientes en ese contexto. La moraleja es que los principios de diseño deben adaptarse a las restricciones del dominio, no al revés.

Otro caso fue un sistema batch de procesamiento nocturno con cero requisitos de mantenimiento a largo plazo (sería reemplazado). El equipo aplicó SOLID religiosamente. El código era hermoso, pero el tiempo de desarrollo fue un 40% mayor que si hubieran escrito funciones simples y acopladas. El cliente no apreció la belleza; solo notó el retraso.

No son reglas absolutas, sino guías

La postura más madura es considerar SOLID como un **conjunto de heurísticas**, no como leyes. Son herramientas en la caja, no la Constitución. Violar SOLID intencionalmente es a veces la decisión correcta, siempre que entiendas las consecuencias. El problema no es violarlos, es violarlos por ignorancia o por pereza sin evaluar el costo futuro.

Cuando decides violar SRP para poner dos responsabilidades juntas porque cambian siempre al mismo tiempo y por la misma razón, esa violación es inocua. Cuando decides no aplicar DIP porque la dependencia concreta es parte de la biblioteca estándar y nunca cambiará, es sensato. La clave está en la conciencia del trade-off.

Cómo decidir cuándo aplicar SOLID

Después de años de prueba y error, he llegado a un criterio simple: aplica SOLID cuando la zona de código cumpla al menos dos de estas condiciones:

- Es parte del núcleo del dominio de negocio (no infraestructura).
- Cambia con frecuencia (más de una vez al mes).
- La desarrollan varios miembros del equipo (necesita ser mantenible por otros).
- Tiene más de una implementación posible actualmente o en el horizonte cercano.
- Necesita ser probada unitariamente sin entorno real.

En caso contrario, el código simple y acoplado puede ser suficiente. Y siempre, siempre, mantén la opción de refactorizar hacia SOLID cuando el costo de no hacerlo supere el costo de hacerlo. La deuda técnica no es pecado si se gestiona conscientemente.

Recursos para profundizar en críticas

Para quienes quieran explorar el otro lado del debate, recomiendo leer:

- "Clean Code" de Robert C. Martin (a favor, pero con matices).
- "Critique of SOLID" de Nikita Popov (crítica técnica desde el mundo PHP).
- "SOLID is not solid" de Sandi Metz (conferencia, aborda cuándo no aplicarlos).
- La sección de "Anti-SOLID" en blogs de programación funcional.

El objetivo no es convertirte en un fanático de SOLID ni en un detractor. Es darte el criterio para saber cuándo sí y cuándo no. Como ocurre con todas las herramientas, el valor no está en el martillo, sino en la habilidad de elegir el martillo adecuado para cada clavo.

Tema 9: Ejercicios prácticos

Ejercicio 1: Identificar violaciones de SRP

Observa la siguiente clase. ¿Cuántas responsabilidades tiene? ¿Qué cambios la afectarían?

```
class Factura:
    def __init__(self, numero, cliente, items):
        self.numero = numero
        self.cliente = cliente
        self.items = items

    def calcular_total(self):
        total = 0
        for item in self.items:
            total += item['precio'] * item['cantidad']
        return total

    def imprimir_factura(self):
        print(f"Factura N° {self.numero}")
        print(f"Cliente: {self.cliente}")
        for item in self.items:
            print(f"{item['descripcion']} - {item['cantidad']} x {item['precio']}")
        print(f"Total: {self.calcular_total()}")

    def guardar_en_bd(self):
        import sqlite3
        conn = sqlite3.connect('facturas.db')
        conn.execute("INSERT INTO facturas VALUES (?, ?, ?)",
                     (self.numero, self.cliente, self.calcular_total()))
        conn.commit()
        conn.close()
```

Tarea: Identifica al menos tres responsabilidades. Propón una refactorización separando cada una en su propia clase.

Solución esperada: Separar en `Factura` (solo datos), `CalculadorTotalFactura`, `ImpresorFactura`, `RepositorioFactura`.

Ejercicio 2: Refactorizar para OCP

El siguiente código viola OCP porque cada nuevo tipo de empleado requiere modificar la clase `CalculadorSalario`.

```
class CalculadorSalario:
    def calcular(self, empleado):
        if empleado.tipo == "desarrollador":
            return empleado.sueldo_base * 1.2
        elif empleado.tipo == "gerente":
            return empleado.sueldo_base * 1.5
        elif empleado.tipo == "vendedor":
            return empleado.sueldo_base * 1.1 + empleado.comisiones
        else:
            return empleado.sueldo_base
```

Tarea: Refactoriza usando el patrón Strategy para que se puedan añadir nuevos tipos sin modificar `CalculadorSalario`. Define una interfaz `EstrategiaSalario` y crea implementaciones concretas.

Solución esperada:

```
from abc import ABC, abstractmethod

class EstrategiaSalario(ABC):
    @abstractmethod
    def calcular(self, empleado):
        pass

class SalarioDesarrollador(EstrategiaSalario):
    def calcular(self, empleado):
        return empleado.sueldo_base * 1.2

class SalarioGerente(EstrategiaSalario):
    def calcular(self, empleado):
        return empleado.sueldo_base * 1.5

class SalarioVendedor(EstrategiaSalario):
    def calcular(self, empleado):
        return empleado.sueldo_base * 1.1 + empleado.comisiones

class CalculadorSalario:
```

```
def __init__(self, estrategia: EstrategiaSalario):
    self.estrategia = estrategia

def calcular(self, empleado):
    return self.estrategia.calcular(empleado)
```

Ejercicio 3: Detectar y corregir violación de LSP

La siguiente jerarquía parece razonable, pero rompe LSP. Explica por qué y corrige el diseño.

```
class Pajaro:
    def volar(self):
        return "Estoy volando"

class Pinguino(Pajaro):
    def volar(self):
        raise NotImplementedError("Los pingüinos no vuelan")

def hacer_volar(pajaro: Pajaro):
    pajaro.volar()
```

Tarea: Rediseña para que **Pinguino** pueda sustituir a **Pajaro** sin romper el código cliente.

Solución esperada: Separar la interfaz en **PajaroVolador** y **PajaroNoVolador**, o usar composición. Una opción:

```
from abc import ABC, abstractmethod

class Ave(ABC):
    pass

class Volador(ABC):
    @abstractmethod
    def volar(self):
        pass

class PajaroVolador(Ave, Volador):
    def volar(self):
        return "Estoy volando"

class Pinguino(Ave):
    def nadar(self):
        return "Estoy nadando"

def hacer_volar(volador: Volador):
    print(volador.volar())
```

Ejercicio 4: Segregar interfaces gordas

La siguiente interfaz obliga a implementar métodos que no siempre corresponden. Refactoriza aplicando ISP.

```
from abc import ABC, abstractmethod

class Trabajador(ABC):
    @abstractmethod
    def trabajar(self):
        pass

    @abstractmethod
    def comer(self):
        pass

    @abstractmethod
    def dormir(self):
        pass

class Robot(Trabajador):
    def trabajar(self):
        print("Robot trabajando")

    def comer(self):
        raise Exception("No come")

    def dormir(self):
        raise Exception("No duerme")
```

Tarea: Crea interfaces separadas (*Trabajable*, *Comible*, *Durmiente*) y haz que *Robot* implemente solo la que necesita.

Solución esperada:

```
class Trabajable(ABC):
    @abstractmethod
    def trabajar(self):
        pass

class Comible(ABC):
    @abstractmethod
    def comer(self):
        pass

class Durmiente(ABC):
    @abstractmethod
    def dormir(self):
```

```
pass

class Robot(Trabajable):
    def trabajar(self):
        print("Robot trabajando")
```

Ejercicio 5: Invertir dependencias (DIP)

El siguiente servicio de notificaciones depende directamente de clases concretas de bajo nivel. Aplica DIP.

```
class EmailSender:
    def send(self, mensaje, destino):
        print(f"Enviando email a {destino}: {mensaje}")

class SMSSender:
    def send(self, mensaje, telefono):
        print(f"Enviando SMS a {telefono}: {mensaje}")

class Notificador:
    def __init__(self):
        self.email = EmailSender()
        self.sms = SMSSender()

    def notificar(self, tipo, destino, mensaje):
        if tipo == "email":
            self.email.send(mensaje, destino)
        elif tipo == "sms":
            self.sms.send(mensaje, destino)
```

Tarea: Introduce una abstracción `Sender` y haz que `Notificador` dependa de ella. Cambia el uso para que sea por inyección.

Solución esperada:

```
from abc import ABC, abstractmethod

class Sender(ABC):
    @abstractmethod
    def send(self, mensaje, destino):
        pass

class EmailSender(Sender):
    def send(self, mensaje, destino):
        print(f"Enviando email a {destino}: {mensaje}")

class SMSSender(Sender):
    def send(self, mensaje, destino):
```



```

        print(f"Enviando SMS a {destino}: {mensaje}")

class Notificador:
    def __init__(self, sender: Sender):
        self.sender = sender

    def notificar(self, destino, mensaje):
        self.sender.send(mensaje, destino)

# Uso
notif = Notificador(EmailSender())
notif.notificar("ana@mail.com", "Hola")

```

Ejercicio 6: Refactorización completa de un mini sistema

Dado el siguiente sistema de gestión de tareas, que viola múltiples principios, refactoriza aplicando SOLID paso a paso.

```

class GestorTareas:
    def __init__(self):
        self.tareas = []

    def agregar_tarea(self, descripcion, prioridad):
        self.tareas.append({"desc": descripcion, "prio": prioridad,
"completada": False})

    def completar_tarea(self, indice):
        if 0 <= indice < len(self.tareas):
            self.tareas[indice]["completada"] = True

    def prioridad_alta(self):
        return [t for t in self.tareas if t["prio"] == "alta"]

    def guardar_en_archivo(self, filename):
        with open(filename, "w") as f:
            for t in self.tareas:
                f.write(f"{t['desc']}|{t['prio']}|{t['completada']}\n")

    def cargar_de_archivo(self, filename):
        self.tareas = []
        with open(filename, "r") as f:
            for linea in f:
                partes = linea.strip().split("|")
                self.tareas.append({
                    "desc": partes[0],
                    "prio": partes[1],
                    "completada": partes[2] == "True"
                })

```

```
def mostrar(self):
    for i, t in enumerate(self.tareas):
        estado = "✓" if t["completada"] else "✗"
        print(f"{i}. [{estado}] {t['desc']} (prioridad: {t['prio']})")
```

Tarea: Identifica al menos cuatro violaciones y refactoriza en clases separadas: **Tarea** (entidad), **RepositorioTareas** (persistencia), **FiltroPrioridadAlta** (lógica de filtrado), y **VistaTareas** (presentación). Asegúrate de que las dependencias apunten a abstracciones.

Solución esperada (esquema, no el código completo):

- Clase **Tarea**: atributos privados, métodos de negocio (**completar**, **esta_completada**).
- Interfaz **RepositorioTareas** con **guardar**, **cargar**, **agregar**, **obtener_todas**.
- Implementaciones concretas: **RepositorioArchivo**, **RepositorioMemoria**.
- Clase **Filtrador** con método **filtrar_prioridad_alta**.
- Clase **Presentador** que recibe una lista de tareas y las imprime.
- El **GestorTareas** original se rompe en pequeños componentes que colaboran mediante inyección.

Ejercicio 7: Diseño desde cero respetando SOLID

Enunciado: Diseña un sistema de cola de impresión. Debe permitir agregar documentos, imprimir en diferentes impresoras (láser, inyección, 3D), y notificar al usuario cuando termine (por email o por mensaje en pantalla). Aplica todos los principios SOLID.

Pasos sugeridos:

1. Define las abstracciones: **Impresora** (con método **imprimir(documento)**), **Notificador** (con método **notificar(usuario, mensaje)**).
2. Implementa impresoras concretas (**ImpresoraLaser**, **ImpresoraInyeccion**, **Impresora3D**).
3. Implementa notificadores concretos (**NotificadorEmail**, **NotificadorPantalla**).
4. Crea la clase **ColaImpresion** que dependa de **Impresora** y **Notificador** (inyectados). Debe tener métodos **agregar_documento**, **procesarCola**.
5. Asegura que añadir una nueva impresora o notificador no requiera modificar **ColaImpresion**.
6. Escribe una pequeña demostración de uso.

Código base sugerido (completar):

```
from abc import ABC, abstractmethod
from collections import deque

class Documento:
    def __init__(self, nombre, contenido):
        self.nombre = nombre
        self.contenido = contenido

class Impresora(ABC):
    @abstractmethod
    def imprimir(self, doc: Documento):
```

```

pass

class Notificador(ABC):
    @abstractmethod
    def notificar(self, usuario, mensaje):
        pass

# Implementaciones aquí...

class ColaImpresion:
    def __init__(self, impresora: Impresora, notificador: Notificador):
        self.impresora = impresora
        self.notificador = notificador
        self cola = deque()

    def agregar_documento(self, doc: Documento):
        self.colas.append(doc)

    def procesarCola(self, usuario):
        while self.colas:
            doc = self.colas.popleft()
            self.impresora.imprimir(doc)
            self.notificador.notificar(usuario, f"Documento {doc.nombre}
impreso")

```

Ejercicio 8: Análisis de código legacy

Toma un fragmento de código real de un proyecto existente (puedes inventar uno). Identifica cuáles principios se violan y escribe un plan de refactorización en no más de 10 pasos. Incluye qué nuevos archivos crearías y cómo cambiarían las pruebas.

Ejemplo de código legacy (típico en sistemas web):

```

class ControladorUsuario:
    def registrar(self, request):
        email = request.POST['email']
        password = request.POST['password']
        # validaciones
        if '@' not in email:
            return "Email inválido"
        if len(password) < 6:
            return "Password débil"
        # guardar en MySQL
        conn = mysql.connector.connect(database="app")
        cursor = conn.cursor()
        cursor.execute("INSERT INTO usuarios (email, pass) VALUES (%s, %s)",
(email, md5(password)))
        conn.commit()
        # enviar email

```

```
smtp = smtplib.SMTP('smtp.gmail.com', 587)
smtp.sendmail('admin@site.com', email, 'Bienvenido')
return "OK"
```

Plan de refactorización sugerido:

1. Extraer validación a una clase **ValidadorUsuario**.
2. Extraer hashing a **HashStrategy** (con implementación MD5, luego se puede cambiar).
3. Extraer persistencia a **RepositorioUsuario** con interfaz.
4. Extraer notificación a **BienvenidaNotificador**.
5. Hacer que **ControladorUsuario** reciba estas dependencias por constructor.
6. Cambiar pruebas para usar mocks.

Recomendaciones para practicar

- **Katas de programación:** ejercita SOLID en katas como "Bowling Game", "Gilded Rose", "Mars Rover". Refactoriza las versiones iniciales que violan principios.
- **Reviews de código:** en tu equipo, señala violaciones durante code reviews y propón refactorizaciones pequeñas.
- **Proyectos personales:** elige un proyecto pequeño que hayas hecho antes y aplícale SOLID. Notarás la diferencia en la facilidad para añadir nuevas funciones.

Los ejercicios aquí planteados son un punto de partida. La verdadera maestría llega cuando, al enfrentarte a un problema, no piensas "¿cómo aplico SOLID?", sino que el diseño desacoplado y mantenible surge de forma natural.

Tema 10: Recursos adicionales

Libros fundamentales

Hay libros que se convierten en compañeros de ruta. No son de lectura única; se vuelven a ellos cada vez que un diseño se siente torcido.

- **"Clean Code: A Handbook of Agile Software Craftsmanship"** – Robert C. Martin. El capítulo sobre clases y funciones toca SRP y OCP de manera práctica. El resto del libro complementa SOLID con nombres significativos, comentarios justos y formato consistente.
- **"Agile Software Development, Principles, Patterns, and Practices"** – Robert C. Martin. Aquí nacieron los principios SOLID como los conocemos hoy. Incluye estudios de caso completos, no solo ejemplos de juguete.
- **"Head First Design Patterns"** – Freeman & Robson. Aunque se centra en patrones, cada capítulo muestra cómo un patrón (Strategy, Template Method, Decorator) ayuda a cumplir OCP o DIP. Lo recomiendo para quienes aprenden mejor con diagramas y ejercicios.
- **"Refactoring: Improving the Design of Existing Code"** – Martin Fowler. SOLID es una meta; la refactorización es el camino. Este libro enseña a transformar código que viola principios en código limpio paso a paso.
- **"Design Patterns: Elements of Reusable Object-Oriented Software"** – Gamma, Helm, Johnson, Vlissides (Gang of Four). Es un clásico denso, pero leer los patrones relacionados (Strategy,

Observer, Abstract Factory) ayuda a ver cómo SOLID se materializa en la práctica.

Cursos y tutoriales en línea

La experiencia ajena ahorra años de tropiezos. Estos recursos han demostrado claridad y profundidad:

- **"SOLID Principles of Object-Oriented Design"** (Pluralsight) – Steve Smith. Un curso completo con ejemplos en C# que se entiende aunque no domines el lenguaje. La sección sobre detección de violaciones es especialmente útil.
- **"SOLID Principles for Python Developers"** (Real Python) – Artículo gratuito con ejemplos específicos del ecosistema Python. Aborda cómo el tipado pato afecta la aplicación de LSP e ISP.
- **"Object Oriented Design"** (University of Alberta, Coursera) – Un curso más académico, pero los módulos sobre Liskov y contratos de diseño son de los mejores que he visto.
- **"SOLID Principles" en YouTube – Derek Banas**. Un video rápido (menos de 15 minutos) con ejemplos en Java. Ideal para repasar antes de una entrevista o una refactorización urgente.

Herramientas de análisis estático

Ningún linter reemplaza el criterio humano, pero las herramientas capturan violaciones obvias que en una revisión manual se escapan.

- **SonarQube / SonarLint**: Detecta clases demasiado largas (sospecha de SRP), alto acoplamiento (violación de DIP) y complejidad ciclomática elevada (posible violación de OCP). La regla `java:S110` (herencia demasiado profunda) alerta sobre jerarquías que pueden romper LSP.
- **Pylint (Python)**: Las reglas `R0902` (too-many-instance-attributes) y `R0904` (too-many-public-methods) son proxies útiles para detectar SRP e ISP violados.
- **PMD**: Para Java, el conjunto de reglas `DesignRules` incluye `UseUtilityClass`, `AvoidDeeplyNestedIfStmts` y `GodClass`. No son SOLID explícitamente, pero los dioses y los condicionales profundos suelen indicar problemas.
- **PHPStan / Psalm (PHP)**: Con niveles máximos, fuerzan a declarar interfaces y tipos, lo que indirectamente promueve DIP. La regla `checkUnusedConstructorDependencies` ayuda a detectar dependencias inyectadas pero no usadas.
- **ArchUnit (Java)**: Permite escribir reglas de arquitectura como `classes().that().resideInAPackage("..domain..").should().onlyDependOnClassesThat().resideInAnyPackage("..domain..", "java..")`. Es la herramienta más cercana a automatizar DIP y proteger fronteras arquitectónicas.

Katas y repositorios para practicar

La teoría se olvida; el músculo de refactorización se gana con repetición deliberada.

- **Gilded Rose Kata**: El problema clásico de refactorización. La versión inicial viola OCP (una larga cadena de `if` por tipo de ítem) y SRP (actualiza calidad, vende, maneja límites). Practicar este kata aplicando Strategy y Template Method es un rito de iniciación.
- **Bowling Game Kata** (Uncle Bob): Pequeña, pero enseña a no anticipar abstracciones (YAGNI) mientras se aplica OCP cuando aparecen los spare y strike.
- **Mars Rover Kata**: Implica comandos y obstáculos. Ideal para practicar DIP (inyectar un detector de obstáculos) y OCP (nuevos comandos sin modificar el rover).

- **Repositorio "solid-examples" en GitHub:** Busca repositorios con nombres como `solid-principles-examples` en tu lenguaje favorito. Revisa el código de otros y crítico mentalmente. ¿Aceptarías ese código en tu equipo?
- **Refactoring.guru:** Sitio web con ejemplos visuales de patrones y principios. La sección "SOLID" incluye diagramas antes/después y código en varios lenguajes (Python, PHP, Java, C++, etc.).

Comunidades y foros

Discutir casos reales con otros profesionales acelera el aprendizaje más que leer diez libros.

- **Software Engineering Stack Exchange:** Busca las etiquetas `solid-principles` y `design-patterns`. Hay discusiones muy matizadas sobre si una violación es aceptable en un contexto dado.
- **Reddit r/softwarearchitecture:** Publicaciones donde se analizan fragmentos de código y se proponen refactorizaciones SOLID. Los comentarios suelen ser críticos pero constructivos.
- **Dev.to / Medium:** Sigue a autores como "Sandro Mancuso", "Yegor Bugayenko" (aunque controvertido, sus posts sobre DIP y OCP son provocadores) y "Martin Fowler". Filtra por fecha para ver aplicaciones modernas.
- **Discord / Slack de comunidades ágiles:** Muchos grupos de programación tienen canales de `#clean-code` o `#architecture`. Plantea un fragmento real de tu trabajo (anonimizado) y pide opinión sobre violaciones de SOLID.

Hojas de referencia rápida (cheatsheets)

Cuando estás en medio de una refactorización, una chuleta mental ayuda:

Principio	Pregunta clave	Señal de violación
SRP	¿Cuántos actores podrían pedir cambios sobre esta clase?	La clase tiene más de 200 líneas o métodos que usan diferentes subconjuntos de atributos.
OCP	¿Para añadir una nueva variante tengo que tocar código existente o solo añadir uno nuevo?	Condicionales que verifican un "tipo" y ejecutan comportamientos distintos.
LSP	¿Puedo pasar una subclase a cualquier función que acepte la base sin alterar su comportamiento esperado?	La subclase lanza <code>NotImplementedError</code> o cambia el estado de manera inesperada.
ISP	¿Hay métodos en la interfaz que algunas implementaciones no necesitan?	Implementaciones vacías o que lanzan excepciones. Clientes que solo llaman a un subconjunto pequeño.
DIP	¿El módulo de alto nivel importa directamente una clase concreta de bajo nivel?	<code>import sqlite3</code> dentro de una clase de dominio. Dependencias construidas con <code>new</code> dentro del constructor.

Patrones que implementan SOLID

A veces ayuda ver SOLID como el "por qué" y los patrones como el "cómo":

- **SRP** → Patrón Facade, Simple Factory.
- **OCP** → Strategy, Template Method, Decorator, Observer.
- **LSP** → Null Object Pattern, Command Pattern (bien aplicado).
- **ISP** → Role Interface, Adapter Pattern (cuando adaptas una interfaz gorda en varias pequeñas).
- **DIP** → Dependency Injection, Abstract Factory, Service Locator (aunque éste último es controvertido).

Un último consejo desde la trinchera

No intentes aplicar SOLID en todo el código de golpe. Elige un módulo que se haya vuelto doloroso de mantener. Identifica la violación más evidente (suelen ser SRP o DIP). Refactoriza esa única clase en dos o tres. Observa cómo las pruebas existentes (si las hay) te guían. Commitea. Al cabo de unas semanas, el síndrome del código frágil empezará a remitir. El resto de principios se irán añadiendo solos porque el código limpio atrae más limpieza.

Y si algún día te ves escribiendo una interfaz con un solo método solo porque "lo dice DIP", haz una pausa y pregúntate si esa abstracción realmente pagará su costo de indirección. A veces la respuesta es no. Y eso también está bien.

Enlaces útiles para empezar hoy mismo:

- [SOLID Principles cheatsheet \(PDF\)](#) – Imprime y pega en tu monitor.
- [Refactoring Gilded Rose kata \(starter code\)](#) – Elige tu lenguaje.
- [SonarLint free plugin](#) – Instálalo en tu IDE y ejecútalo sobre tu código legacy. Los warnings serán tu lista de tareas pendientes.

Ahora, a escribir código que no avergüence al equipo del próximo año.